MECC: Single-GPU MoE Inference with Compiler-Guided Three-Tier Expert Caching

Anonymous Authors

***Abstract*—Single-GPU mixture-of-experts inference requires SSD reads when the expert pool exceeds the available HBM and host working set. Weight delivery then delays token computation. We present MECC, a compiler-guided three-tier expert cache. An independent route profile determines initial placement. Runtime promotion and demotion retain reused experts and track working-set changes. A dependency-aware schedule overlaps weight delivery with computation. These mechanisms preserve expert selection and FP16 weights. Hypothetical results show 3.0–3.3× speedup over unoptimized streaming. One target point uses a single GPU at eight seconds per token, compared with eight resident GPUs at one second per token.**

***Index Terms*—Mixture of experts, single-GPU inference, expert offloading, three-tier caching, hardware/software co-design.**

I. Introduction

Mixture-of-experts (MoE) models select a subset of experts for each token [1], [2]. A single GPU can execute these experts without keeping the full model in high-bandwidth memory (HBM). This deployment reduces the number of GPUs assigned to a task. It requires a mechanism to deliver each selected expert before computation.

Shared servers assign each service a host-memory allowance. This allowance may cover only part of the expert pool. When the combined HBM and host working set is too small, selected experts are fetched from SSD. The GPU waits for these transfers. We study how to reduce this waiting under fixed task-level memory budgets.

Prior work uses CPU/GPU orchestration [3], memory-constrained planning [4], and SSD-native delivery [5]. Learned replacement [6] and domain-guided residency [7] further exploit routing information. We compare placement, retention, and transfer scheduling under common physical memory caps. Section VI details the related mechanisms.

Three coupled decisions govern expert delivery. **First, capacity has to be distributed across layers.** Uniform slots can leave some layers repeatedly reading from SSD. **Second, HBM eviction has to preserve later reuse.** A fixed CPU pool may discard useful experts, while independent tier caches can retain duplicate copies. **Third, residency has to track request changes.** An initial profile covers startup demand; online observations update that placement as the working set changes.

Our insight is to treat the route profile as an initial residency prior and actual requests as online corrections. We implement this division in MECC, a cache across HBM, CPU pinned memory, and SSD. The compiler assigns expert rankings and per-layer capacities from an independent profile. The runtime promotes requested experts and demotes HBM victims into the host working set. A copy schedule overlaps transfers with independent work and enforces weight readiness before computation. Figure 1 shows their interfaces.

The contributions are: (1) a compiler producing budgeted residency plans; (2) a runtime coordinating retention and transfer dependencies while preserving selected experts and precision; and (3) an evaluation separating component gains from the single-/multi-GPU resource tradeoff. Hypothetical results target 1.25–1.42× speedup over the local SSD-native adapter. All numerical outcomes are authored scenarios.

II. Deployment Model and Challenges

A. Single-GPU Execution with Bounded Host Memory

Consider L MoE layers. Layer ℓ contains N_ℓ experts and selects k_ℓ per token. Expert $e = (\ell, j)$ is an immutable weight object of size s_e. The GPU also stores dense weights, activations, the KV cache, and temporary buffers. The expert-residency allowance is B_{H}; the total task-level host allowance is B_{C}. Host mirrors and staging buffers count toward B_{C}.

The target configuration requires SSD reads because the expert pool exceeds the available HBM and host working set. We verify this condition through cold-read traffic. Models that fit in host memory serve as two-tier controls. Models that fit in HBM need no expert offloading.

An HBM hit uses resident weights. A CPU hit requires host-to-device (H2D) transfer. An SSD miss adds an SSD-to-host read. We record the expert's tier when demand arrives, before servicing the request. Separate counters record physical SSD and transfer traffic, including prefetches.

B. Challenge 1: Allocate Capacity across Layers

The value of a cache slot depends on the layer's request distribution and miss cost. A small working set needs few slots. A broader working set can cause repeated cold reads under the same allocation. This difference motivates moving capacity between layers while holding total memory fixed.

Figure 2(a) compares uniform and planned allocations on a held-out stream. Total slots, replacement, precision, and requests are fixed. The hypothetical plan assigns more slots to layers with higher cold-read pressure. Some donor layers incur more misses; the aggregate miss share falls.

C. Challenge 2: Preserve Reuse after HBM Eviction

An expert can be requested again soon after HBM eviction. A fixed CPU pool retains only its predefined members. Independent tier caches may instead retain copies of the same expert. These policies differ in both reuse coverage and physical duplication.

Figure 2(b) compares a fixed CPU pool, independent LRUs, and coordinated retention. The hypothetical results favor retention across tiers. Unique coverage, duplicate bytes, and migration traffic distinguish the causes. An exclusive global-LRU reference controls for combined cache capacity. Figure 3 illustrates the state changes.

D. Challenge 3: Initialize Residency and Track Changes

A profile supplies access information before the first request. Online history tracks changes after service begins. Fixed placement can become stale when the working set shifts. An online policy without

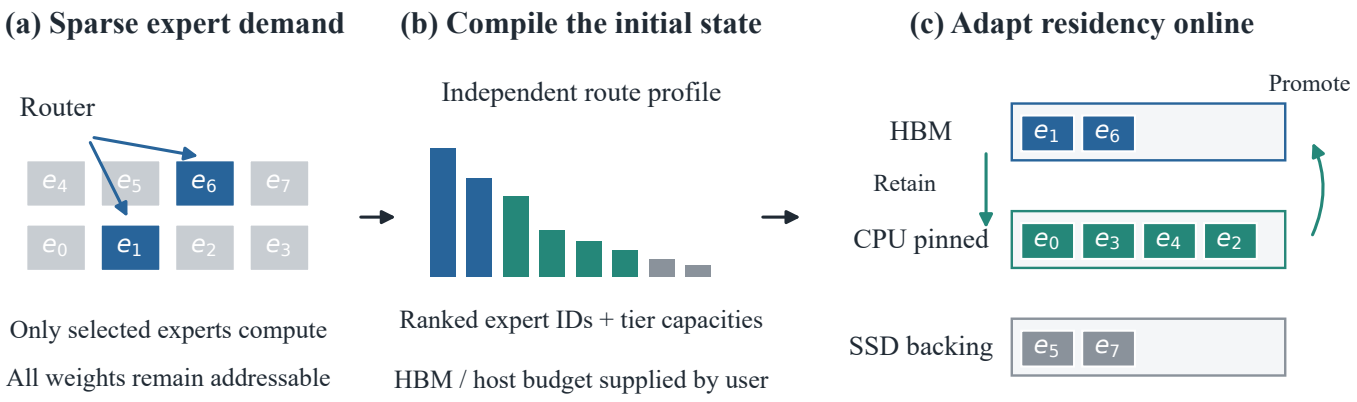


Fig. 1. MECC overview. (a) The router selects experts for computation. (b) An independent profile determines initial residency. (c) Promotion and demotion update the HBM and CPU working sets. SSD holds the backing weights. Expert IDs and weights remain unchanged. Schematic.

a profile first has to discover that working set. Their combination targets startup misses and later adaptation.

We cross profile/random initialization with static/adaptive residency. Figure 2(c) shows hypothetical window-level miss shares across a shift. Initialization helps the early windows; online updates respond to the shift. The full comparison includes profile cost, a no-shift control, uniform access, and mismatched profiles.

III. MECC Design

A. Compiler: Profile-Guided Placement

The compiler takes model metadata, memory budgets, delivery costs, and an independent route profile. The profile supplies request probability p_e for each expert. For equal-size experts within a layer, a discrete Zipf model summarizes the ranked probabilities:

$$p_{\ell,r} = \frac{r^{-\alpha_\ell}}{H_{N_\ell}(\alpha_\ell)}, \qquad H_n(\alpha) = \sum_{r=1}^{n} r^{-\alpha}. \tag{1}$$

Each rank retains its profiled expert ID. The exponent α_ℓ describes access concentration. The placement objective can also use empirical probabilities directly. Neither representation encodes request order.

Let $x_{e,t} \in \{0,1\}$ indicate the primary tier of expert e. The tiers are HBM, CPU, and SSD, denoted by H, C, and S. The compiler reserves buffer and mirror space before assigning residency budgets $\bar{B}_t$. Given delivery cost $T_t(s_e)$, it models

$$\begin{aligned} \min_x \quad & \sum_e p_e \sum_t x_{e,t} T_t(s_e), \\ \text{s.t.} \quad & \sum_t x_{e,t} = 1, \quad \sum_e s_e x_{e,t} \le \bar{B}_t \quad (t \in \{\mathrm{H},\mathrm{C}\}). \end{aligned} \tag{2}$$

The affine model $T_t(s) = a_t + s/b_t$ represents fixed overhead and effective bandwidth. Calibration uses the target machine. Equation 2 estimates static delivery cost under the profile. Runtime locality, migration, and overlap are evaluated separately.

Proposition 1: Assume equal-size objects, fixed request probabilities, independent tier capacities, and $T_\mathrm{H} \le T_\mathrm{C} \le T_\mathrm{S}$. An optimal static placement fills HBM with the most probable objects, CPU with the next most probable objects, and SSD with the remainder.

Proof sketch. Suppose $p_i > p_j$ and object i occupies the slower tier. Exchanging the two objects changes expected cost by $(p_i - p_j)(T_\text{fast} - T_\text{slow}) \le 0$. Repeated exchanges produce ranked placement. Filling an empty faster-tier slot also cannot increase cost. The result applies to the stated static model.

The multilayer implementation allocates slots by marginal coverage. Under the Zipf model, a slot covering rank r adds

$$\Delta_\ell(r) = \frac{r^{-\alpha_\ell}}{H_{N_\ell}(\alpha_\ell)}. \tag{3}$$

The planner allocates HBM slots first. It then allocates CPU slots among the remaining ranked experts. It checks total budgets and per-layer minima before emitting a plan. This sequential procedure is a multilayer heuristic. The static proposition does not establish its global optimality or cover heterogeneous expert sizes.

The plan records per-layer capacities, ranked expert IDs, initial tiers, and modeled hit fractions. Model and profile identifiers track its inputs. If profiling is unavailable, the loader uses a labeled natural-order fallback. It initializes the runtime from the chosen plan.

B. Runtime: Cross-Tier Retention

Each layer has ordered HBM and CPU residency maps. An HBM hit updates recency. A CPU hit removes the expert from CPU residency and promotes it to HBM as most recently used. An SSD miss reads the weights into staging before H2D transfer. When HBM is full, its least-recently used expert is demoted to CPU. CPU pressure evicts the least-recently used host resident. The SSD backing weights remain available.

Figure 3 starts with HBM $[e_0, e_1]$ and CPU $[e_2, e_3, e_4]$. Requesting e_2 promotes it and demotes e_0. The next request for e_0 hits CPU. This example uses exclusive logical residency. An implementation can keep a CPU mirror to avoid device-to-host (D2H) transfer on eviction. The mirror consumes host memory and is charged separately.

Exclusive retention coordinates the two tiers. In the controlled object model, its SSD miss sequence matches global LRU of capacity $K_\mathrm{H}+K_\mathrm{C}$ with the corresponding initial stack. This provides a reference for combined coverage. Static and adaptive policies can each win on particular traces; the workload comparisons determine their behavior here.

C. Delivery: Dependency-Aware Overlap

A dedicated copy stream transfers pinned host weights. Each transfer records a readiness event. The computation stream waits for the event before executing the expert. The source buffer remains live

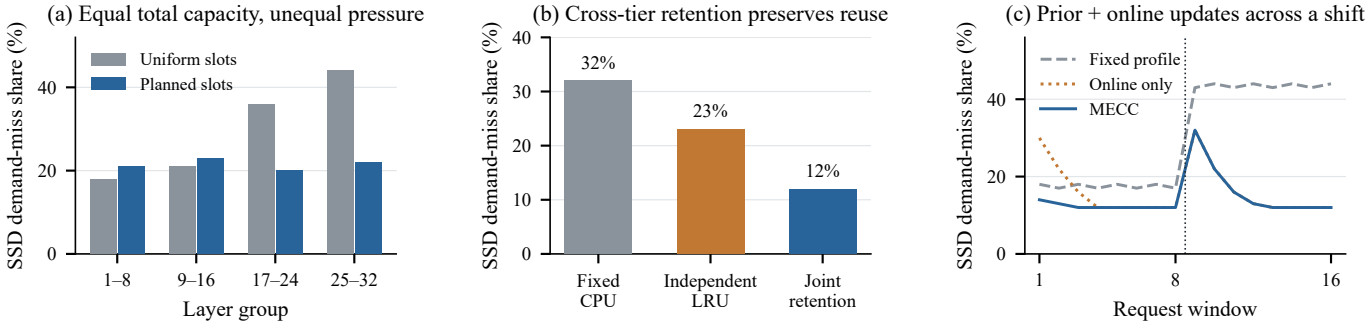


Fig. 2. Deployment challenges. (a) Layer allocation at fixed total capacity. (b) Retention rules under the same initial profile. (c) Startup and adaptation across a working-set shift. **All values and request windows are hypothetical.** The plots do not use measured MoE routes.

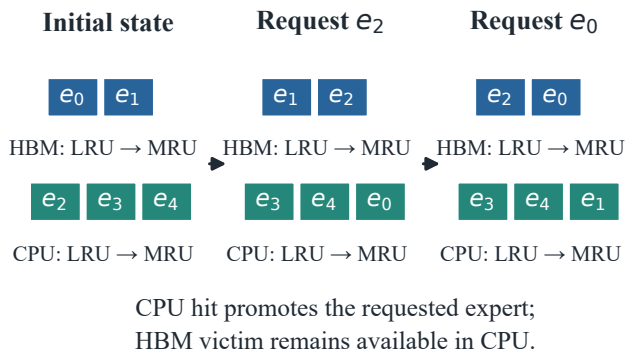


Fig. 3. Cross-tier retention with two HBM slots and three CPU slots. Lists run from LRU to MRU. A CPU hit promotes the requested expert and makes room for the HBM victim. Mirrors and staging consume additional physical space. Schematic.

until the copy completes. Resident weights remain live until their last consuming kernel finishes. These lifetime rules also apply when loading and dispatch use separate iterators.

The schedule overlaps known upcoming transfers with independent work. Replacement comparisons disable predictive prefetch. The overlap ablation applies the same transfer machinery to matched baselines and fixes admission, residency, and requests. It isolates exposed transfer time from cache-policy effects.

The design preserves weights, selected experts, kernels, and aggregation order. Under these conditions, changing residency preserves the forward computation. The implementation also needs correct synchronization. We compare synchronous and asynchronous paths on identical inputs using weight hashes, route selections, logits, tokens, and perplexity.

IV. Implementation

Model adapters map layer/expert IDs to contiguous weight objects. The SSD store serializes them at the original precision. Residency policies share the loader, kernels, and instrumentation. CPU state-machine checks compare policy transitions; GPU tests check execution and synchronization.

Memory accounting deduplicates allocations by storage identity. GPU use includes dense weights, resident experts, KV/activations, and buffers. Host use includes residents, mirrors, pinned staging, pageable buffers, and other weight copies. The hypothetical target uses 70 GB on an 80 GB GPU and 40 GB under a 40 GB task host cap. Host use comprises 28 GB of residents, 6 GB of mirrors, 4 GB of staging, and 2 GB of other allocations.

Initialization, warmup, prefill, decode, and quality evaluation have separate timers and traffic counters. TPOT divides decode time by measured token intervals. TTFT covers prefill and the first output token. Model loading is reported separately unless a cold-start measurement includes it. Demand-source counters describe residency at arrival. Physical SSD/H2D/D2H counters describe the work performed during the same window.

V. Evaluation

Hypothetical evaluation. All results, repeat variation, quality checks, and memory peaks below are authored targets. They are not measurements.

A. Experimental Methodology

Models and workloads. The scenario uses Mixtral-8x7B and Mixtral-8x22B [2], [8] with question-answering and code prompts. One A100 provides 80 GB of physical HBM. The nominal expert allowance is 50 GB, subject to total-peak checks. The task host cap is 40 GB. Profile and evaluation splits are independent. Main comparisons use batch one, 128 input tokens, 128 output tokens, and deterministic decoding. Dataset, model, and input revisions remain to be frozen for execution.

Unoptimized single-GPU baseline. The streaming baseline executes the same selected FP16 experts on one GPU. It shares the model, tokenizer, kernels, cold-read semantics, and task caps with MECC. It uses demand-driven staging with profile placement, dynamic CPU retention, and overlap disabled. This comparison measures the effect of the complete optimization stack.

Cache baselines. Three-tier LRU controls for basic caching. RF and SSD-native are hypothetical local mechanism adapters. RF uses a recency/frequency heuristic; FlashMoE [6] learns a predictor from these signals. SSD-native represents three-tier residency motivated by SSD-LLaMA [5]. Neither establishes a reproduction of the cited system. Versioned external runs, including MoE-CORE [7], remain separate experiments.

Multi-GPU reference. The resident reference partitions the full FP16 model across two, four, or eight A100s where capacity permits. It uses matched inputs and decoding without SSD delivery.

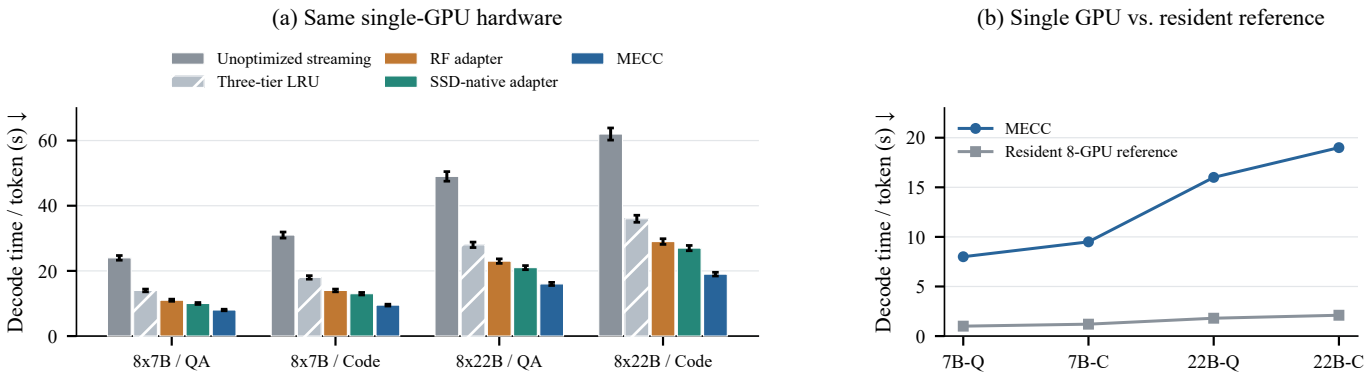


Fig. 4. **Hypothetical performance.** (a) Single-A100 policies with matched task budgets. RF and SSD-native denote local adapters, not published-system measurements. Error bars show authored ±3% spread. (b) Single-GPU MECC versus an eight-A100 resident reference. This comparison uses different GPU budgets.

Its resource record includes GPU count, parallelism, interconnect, software revision, and memory peak. The eight-GPU point is an authored scenario reference, independent of the cited systems.

Metrics. We report TPOT, TTFT, throughput, demand-miss share, SSD/H2D/D2H bytes, and physical memory peaks. GPU-time comparisons use the same workload. Error bars represent three authored values at 0.97, 1.00, and 1.03 times each mean. They show hypothetical spread, not measured uncertainty.

B. Latency under Matched Single-GPU Budgets

Figure 4(a) fixes one A100 and the task host cap across four model/workload combinations. Scenario TPOT is 8.0, 9.5, 16.0, and 19.0 s for MECC, versus 24, 31, 49, and 62 s for streaming. These values correspond to 3.00×, 3.26×, 3.06×, and 3.26× speedup. The comparison measures the full placement, retention, and delivery stack. Section V-D separates those effects.

The SSD-native adapter takes 10, 13, 21, and 27 s per token. Relative to this cache baseline, MECC gains 1.25–1.42×. The three-tier-LRU comparison further separates basic caching from profile placement and coordinated delivery.

Scenario TTFT is 60/68/96/112 s for MECC and 130/155/230/280 s for streaming. These values describe first response. TPOT describes later token intervals. Initialization is charged separately in the cost analysis below.

C. Latency versus GPU Allocation

Figure 4(b) gives eight-GPU TPOT of 1.0/1.2/1.8/2.1 s. The resident reference reduces latency at a higher GPU allocation. The single-GPU point serves applications whose latency allowance permits this tradeoff.

Table I isolates the first deployment point. MECC uses one GPU at 8 s/token; the resident reference uses eight GPUs at 1 s/token. Both use 8 GPU-s/token. This point exchanges GPU allocation for latency at equal nominal GPU-time. Energy and monetary cost also depend on host work, SSD activity, utilization, interconnect, and billing.

D. Component and Mechanism Analysis

Figure 5 adds components to streaming on the first workload. Profile placement reduces scenario TPOT from 24 to 18 s. Retention reduces it to 11 s. Overlap reduces it to 8 s. Each change measures a component within that stack. Component interactions require additional leave-one-out and factorial comparisons.

TABLE I
HYPOTHETICAL 8X7B DEPLOYMENT POINT, NOT MEASURED.

Metric	Single-GPU MECC	Resident reference
Allocated A100 GPUs	1	8
Decode time / token (s)	8	1
GPU-seconds / token	8	8

SSD demand-miss shares are 46%, 33%, 20%, and 20%. Placement and retention reduce cold demand. Overlap keeps the same miss share and reduces exposed waiting. The 12% target in Figure 2(b) belongs to a separate controlled trace.

Mechanism controls hold requests fixed. The placement comparison varies only initialization and charges initial transfers. The retention comparison records duplicate bytes under each policy. Within-phase shuffling preserves expert histograms and tests sensitivity to order. The shift scenario characterizes an authored workload; it provides no measured evidence of natural MoE drift.

E. Resource Sensitivity, Cost, and Quality

Figure 6(a) increases the host cap from 16 to 48 GB on a fixed GPU. Scenario TPOT falls from 16 to 7 s for MECC and from 27 to 12 s for conventional LRU. The paired curves compare residency at each task allowance. Physical use includes the mirrors and buffers defined in Section IV.

Figure 6(b) varies GPU count for 8x7B question answering. The resident configuration does not fit on one GPU. Its two-, four-, and eight-GPU targets are 2.8, 1.65, and 1.0 s/token. MECC targets 8.0, 3.9, 2.1, and 1.25 s/token on one through eight GPUs. The comparison includes the smallest feasible resident configuration and exposes the latency–resource frontier.

Figure 6(c) adds initialization to decode time. Initialization costs 30 s for MECC and 12 s for the SSD-native adapter. Decode costs 8 and 10 s/token, respectively. The totals are equal at nine output tokens; MECC is lower from ten tokens onward. This illustration covers initialization plus decode. A full request-cost analysis additionally includes model loading, profile generation, and prefill where they are not already charged.

The complete sensitivity matrix also varies HBM capacity, input/output lengths, batch size, and profile coverage or mismatch. Leave-one-out controls separate zero-copy eviction, pinning, batch

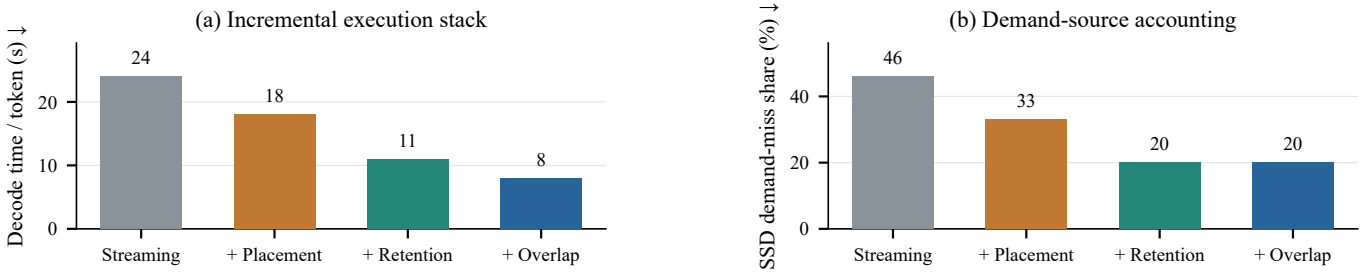


Fig. 5. **Hypothetical ablation.** (a) TPOT after adding each component. (b) SSD demand-miss share. The overlap-only step fixes admission, residency, and requests. Its miss share stays at 20%; exposed transfer time decreases.

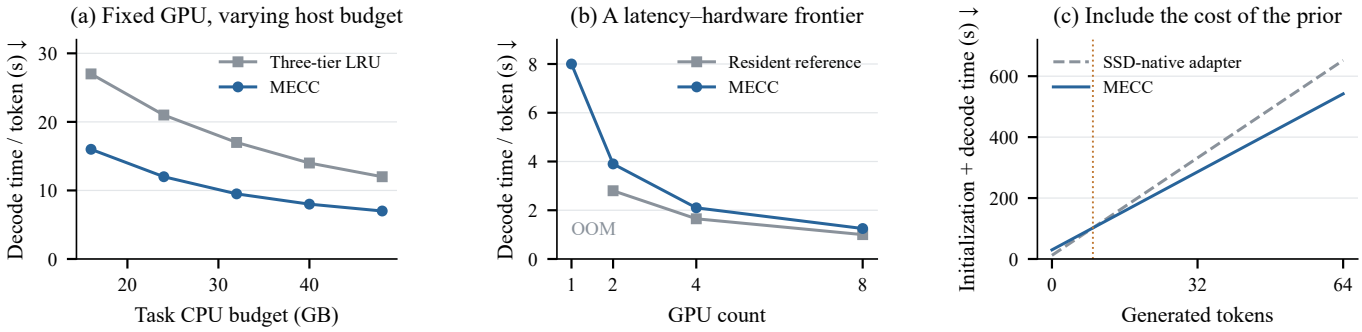


Fig. 6. **Hypothetical sensitivity.** (a) TPOT versus total task host cap. (b) GPU scaling on 8x7B question answering; one-GPU residency does not fit. (c) Initialization plus decode. An 18 s initial overhead and a 2 s/token saving yield equal total time at nine tokens. Additional loading, profiling, and prefill costs are outside this illustration.

H2D, and prefetching. A reverse resource comparison fixes a TPOT limit and finds each policy's minimum feasible host cap. These cases remain planned; the current curves cover only the displayed scenario points.

The quality targets are identical weight hashes and expert selections, zero logit difference from deterministic synchronous execution, and zero perplexity difference on a fixed corpus. These checks remain hypothetical. Nondeterministic kernels require prespecified error tolerances and token comparisons. Weight checks and execution checks test separate properties.

VI. Related Work

Offloading. Fast MoE offloading [9] and SwapMoE [10] study memory-constrained serving. Fiddler [3] initializes GPU residency from expert popularity and executes other experts on CPU. MoE-Lightning [4] searches batching, execution, and weight/KV placement under CPU/GPU memory constraints. Its search excludes disk variables; MECC targets SSD demand.

SSD-based serving. SSD-LLaMA [5] combines expert-pack I/O, lossless transport, and RAM/VRAM caches. Frequency and recent use guide VRAM admission; transient experts can bypass retention. RAM uses LRU. FlashMoE [6] trains a feed-forward distance predictor from normalized recency/frequency signals and Belady-derived labels. Ordinary RF or LRU omits these mechanisms.

Coordinated residency. MoE-CORE [7] uses hit-rate curves for layer allocation, domain profiles for initialization, and routing history/scores for replacement. Optional expert substitution is separate from its exact path. ZipMoE [11] combines lossless bit-field compression, compression-aware caching, and cache-affinity scheduling. MECC evaluates uncompressed initialization and cross-tier retention under matched expert semantics and physical caps.

The scope is single-GPU serving with SSD demand under task-level memory caps. The hypothetical local-adapter comparisons do not establish superiority over external systems. Submission requires replacing all scenario outcomes with measurements.

VII. Conclusion

MECC addresses SSD weight delivery during single-GPU MoE inference. A compiled profile assigns initial residency. Online retention updates the working set, and dependency-aware transfers deliver selected FP16 experts. The hypothetical evaluation separates latency gains at fixed single-GPU budgets from the latency paid to reduce GPU allocation. The resource, mechanism, and quality comparisons define the remaining measurements.

Content Generated by AI

OpenAI Codex (GPT-6) assisted with text in all sections, figure-generation code, and hypothetical outcomes. Human authors are responsible for verifying the content, implementation, references, and replacement measurements before submission.

References

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[2] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[3] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models," in *International Conference on Learning Representations*, 2025.

[4] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

[5] F. Liang, Y. Shen, J. Hu, J. Xu, H. Gao, M. Xu, and Z. Meng, "SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC," *arXiv preprint arXiv:2609.18110*, 2026.

[6] B. Kim, J. Lee, D. Han, H.-J. Yoo, and S. Kim, "FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices," *arXiv preprint arXiv:2601.17063v2*, 2026.

[7] K. Yang, Y. Gao, X. Li *et al.*, "MoE-CORE: Coordinated expert offloading and residency for memory-constrained MoE inference," *arXiv preprint arXiv:2610.01950*, 2026.

[8] Mistral AI, "Cheaper, better, faster, stronger," Official Mixtral 8x22B release, 2024. [Online]. Available: https://mistral.ai/news/mixtral-8x22b

[9] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[10] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget," *arXiv preprint arXiv:2308.15030*, 2023.

[11] Y. Yang, Y. Zhao, P. Yang, S. Wang, and Z.-H. Zhou, "ZipMoE: Efficient on-device MoE serving via lossless compression and cache-affinity scheduling," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 147 261–147 284.

HYPOTHETICAL RESULTS DESIGN DRAFT — NOT MEASURED — NOT FOR SUBMISSION